



SAPIENZA
UNIVERSITÀ DI ROMA

“SAPIENZA” UNIVERSITY OF ROME

FACULTY OF INFORMATION ENGINEERING,
COMPUTER SCIENCE AND STATISTICS

DEPARTMENT OF COMPUTER, CONTROL
AND MANAGEMENT ENGINEERING

MASTER’S DEGREE IN ARTIFICIAL INTELLIGENCE AND ROBOTICS

Autonomous And Mobile Robotics

Lecture notes

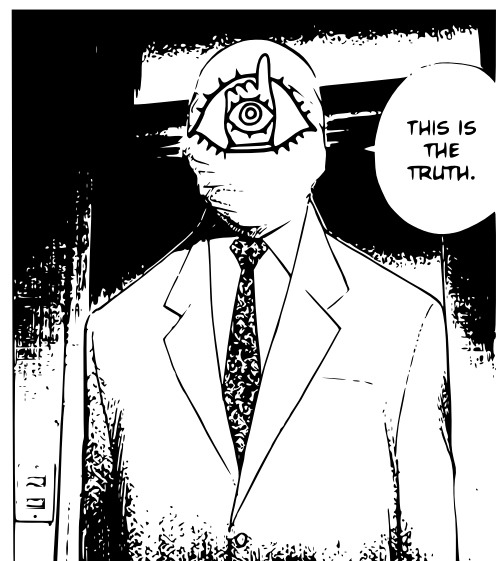
Author

Marco Casu

<https://github.com/CasuFrost>

October 4, 2026

This document summarizes and presents the topics for the Autonomous And Mobile Robotics course for the Master's degree in Artificial Intelligence and Robotics at Sapienza University of Rome. The document is free for any use. If the reader notices any typos, they are kindly requested to report them to the author.



CONTENTS

1	Wheeled Mobile Robots	3
1.1	Configuration Space	3
1.1.1	Topology of a Space	4
1.1.2	Distance in the Configuration Space	6
1.2	Mechanics of Mobile Robots	8
1.2.1	Main Kinematic Structures	9
1.3	Constraints and Mobility	9
1.3.1	Geometric Constraints	10
1.3.2	Kinematic Constraints	11
1.3.3	Non-Holonomic Constraints	12

CHAPTER

1

WHEELED MOBILE ROBOTS

1.1 Configuration Space

We can define a robot as a collection of n_b bodies moving in an operative space, denoted W and called the **workspace**. W can be $\mathbb{R}^2$ (if the robot moves on a plane, like a planar 2R) as well as $\mathbb{R}^3$, if the robot moves in the space. For example, an elbow-type spatial manipulator is a system of $n_b = 3$ rigid bodies moving in $W = \mathbb{R}^3$. A quadrotor is a single body ($n_b = 1$) moving in the same workspace $W = \mathbb{R}^3$.

To describe the complete pose of a planar 2R manipulator, we might use the position vector and roll-pitch-yaw angles for the two bodies, which requires a total of 12 parameters for the whole system, despite that, we know that 2 variables are enough to describe the whole configuration of the system.

Definition 1.1.1 : Configuration

The configuration of a robot is a **minimum** set of parameters that allows one to specify the posture^a of the robot in the workspace.

^awith posture, we indicate the position of all the points of the robot.

The configuration of a robot can be defined in several ways, and a choice of a set of parameters is not unique, however, each robot always has a *minimum number* of parameters. We usually denote $\mathbf{q}$ the configuration vector:

$$\mathbf{q} = \begin{pmatrix} q_1 \\ \vdots \\ q_n \end{pmatrix} \quad (1.1)$$

in our context, it usually describes the Cartesian coordinates of a single body robot, or the angular coordinates of the body parts. The Cartesian coordinates have values in a Euclidean space, as $\mathbb{R}^2$ or $\mathbb{R}^3$, while the angular coordinates take values in a non-Euclidean space, such as the special orthogonal group $SO(3)$, the group of all the orthogonal 3×3 matrices with determinant $+1$, this group describes the orientation of a body in a 3D Euclidean space.

Definition 1.1.2 : Configuration Space

The configuration space, usually denoted $\mathcal{C}$, is the set of all possible values of the configuration of a robot. The *dimension* of $\mathcal{C}$ is the number n of parameters (generalized coordinates) in $\mathbf{q}$.

Let's see an example: consider a disk robot that moves on a plane, that can also rotate about its vertical axis to point towards a desired direction, shown in Figure 1.1.

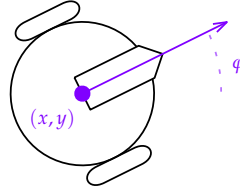


Figure 1.1: Disc robot with a tick in $\mathbb{R}^2$.

Its configuration is given by two Cartesian coordinates (x, y) and a value $\varphi \in SO(2)$ representing the angle between the tick and the horizontal axis. Hence, its configuration space is:

$$\mathcal{C} = \mathbb{R}^2 \times SO(2) = SE(2) \quad (1.2)$$

and it's not Euclidean, despite being called *special Euclidean group*. A polyhedral robot that can freely move in space is described by three Cartesian coordinates (x, y, z) and three roll-pitch-yaw angles (α, β, γ) describing its orientation, hence the configuration space is $SE(3)$:

$$\mathcal{C} = \mathbb{R}^3 \times SO(3) = SE(3). \quad (1.3)$$

We have seen in previous courses of robotics that the number of generalized coordinates of an n -link serial manipulator is exactly n .

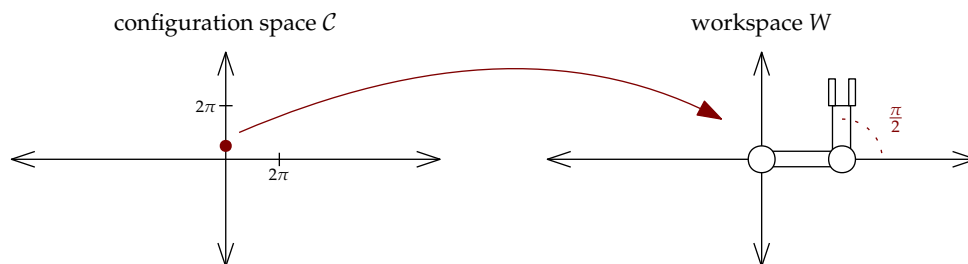
1.1.1 Topology of a Space

We are interested in describing the configuration space of a given manipulator, since it's not Euclidean (in most cases), we would like to have an idea of how to navigate this space. Before defining formally what a *topology* is, we give some examples.

A 2R planar manipulator can be described by two angles, hence two values in $SO(2)$, so its configuration space is

$$\mathcal{C} = SO(2) \times SO(2) = SO(2)^2 \quad (1.4)$$

Let's try to represent graphically $\mathcal{C}$. We might represent a configuration as a point in the Cartesian plane:



The problem of this representation is that different configurations $\mathbf{q} \in \mathcal{C}$ map to the same posture of the robot in W , as shown in Figure 1.2.

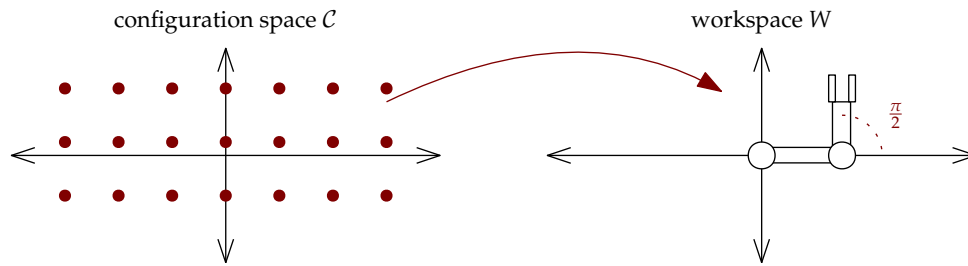
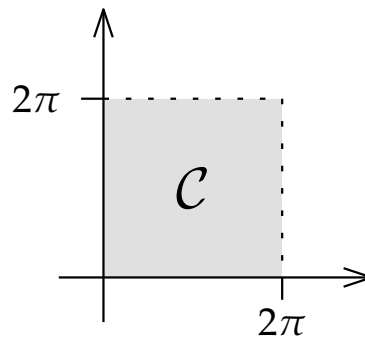
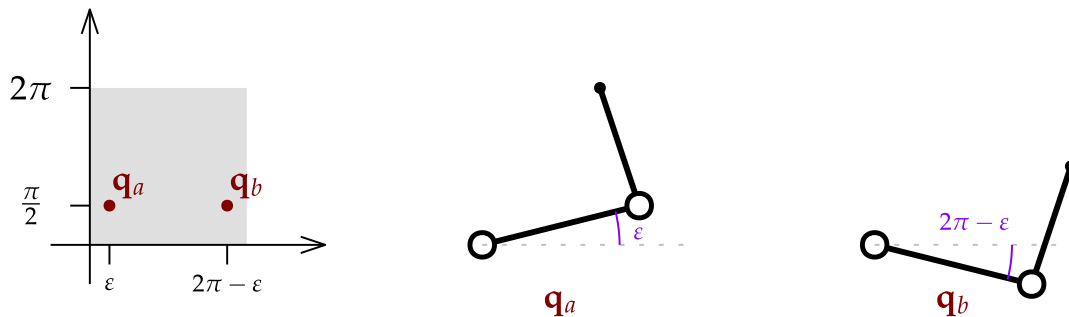


Figure 1.2: Different configurations map to the same posture.

To overcome this problem, we might try to represent the configuration space as a square in the plane $[0, 2\pi) \times [0, 2\pi)$:



This representation also presents a problem: The distance between two configurations in $\mathcal{C}$ does not represent the true distance between two postures in the workspace: Two very distant configurations $\mathbf{q}_a$ and $\mathbf{q}_b$ in $\mathcal{C}$ can be mapped to two very close postures in W :



This representation is said to be **not topologically correct**. To give a correct representation of $\mathcal{C}$ we need to introduce the concept of **manifold**. We will not give a formal and complete definition (as given in topology or differential geometry courses).

Definition 1.1.3 : Manifold (loose definition)

A **manifold** is a topological space that is locally homeomorphic to $\mathbb{R}^n$.

As we should know from previous robotics courses, the configuration space of a 2R is a torus, a manifold that is locally homeomorphic to $\mathbb{R}^2$, and can be visualized in the 3D space through an embedding as shown in Figure 1.3.

$$\mathcal{C} = SO(2) \times SO(2)$$

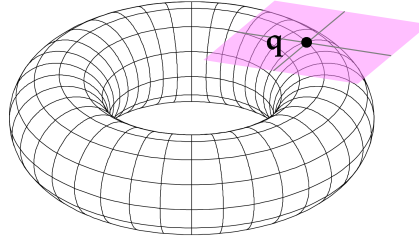


Figure 1.3: 3D representation of a torus.

1.1.2 Distance in the Configuration Space

Definition 1.1.4 : Distance in the Configuration Space

Given a configuration space $\mathcal{C}$, the **distance** is a function $d : \mathcal{C} \times \mathcal{C} \rightarrow \mathbb{R}$ such that, for each $\mathbf{q}_a, \mathbf{q}_b, \mathbf{q}_c$ in $\mathcal{C}$ we have:

$$d(\mathbf{q}_a, \mathbf{q}_b) \geq 0 \quad (1.5)$$

$$d(\mathbf{q}_a, \mathbf{q}_b) = 0 \iff \mathbf{q}_a = \mathbf{q}_b \quad (1.6)$$

$$d(\mathbf{q}_a, \mathbf{q}_b) = d(\mathbf{q}_b, \mathbf{q}_a) \quad (1.7)$$

$$d(\mathbf{q}_a, \mathbf{q}_b) \leq d(\mathbf{q}_a, \mathbf{q}_c) + d(\mathbf{q}_c, \mathbf{q}_b). \quad (1.8)$$

We know how the distance between two vectors $\mathbf{u}, \mathbf{v}$ in $\mathbb{R}^n$ is given by $\|\mathbf{u} - \mathbf{v}\|$. The distance between two points is the *shortest path* between them, for a general manifold, this path is not always a straight line as for the Euclidean space.

Definition 1.1.5 : Geodesic

For a general manifold, the shortest path between two points is called **geodesic**, and the distance between the points is the length of their geodesic.

For example, consider the shortest path between two points on the surface of a sphere (a 2-manifold): the geodesic lies on the great circle (the circle with the biggest radius that passes through the two points), as shown in Figure 1.4. In general, for an arbitrary manifold, it is hard to determine the geodesic between two points.

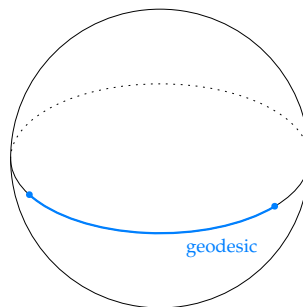


Figure 1.4: Geodesic on a sphere.

In the robotics context, we want to define the distance between two postures of a robot, this distance is defined in the workspace W , and to define that we don't need to specify directly the distance between their respective configurations in $\mathcal{C}$ (it would be harder to find the geodesic on $\mathcal{C}$).

Given a robot and its current configuration $\mathbf{q}$, we can define two functions:

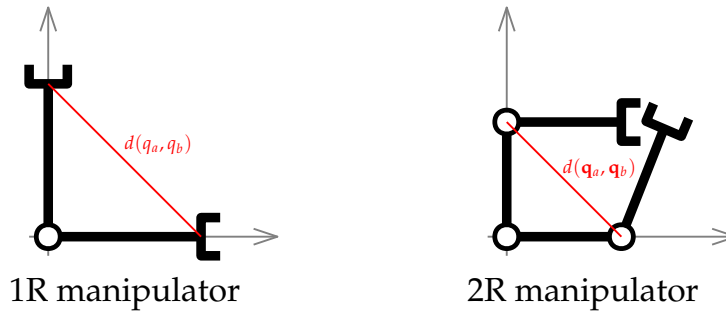
- $B(\mathbf{q})$ is a function that assigns to $\mathbf{q}$ the volume (exact region) of the workspace that the robot occupies when it is in the configuration $\mathbf{q}$, so it assigns to each configuration $\mathbf{q} \in \mathcal{C}$ a set of points in $\mathbb{R}^3$.
- $\mathbf{p}(\mathbf{q})$ is a function that assigns to $\mathbf{q}$ the exact position of a certain point of the robot when it's in the configuration $\mathbf{q}$.

For example, we might let $\mathbf{p}$ be the center of mass of the manipulator, and observe how it changes by varying $\mathbf{q} \in \mathcal{C}$. One simple way to define a distance metric is to consider the maximum Euclidean distance between two points of the manipulator in two different configurations:

$$d(\mathbf{q}_a, \mathbf{q}_b) = \max_{\mathbf{p} \in \text{robot}} \|\mathbf{p}(\mathbf{q}_a) - \mathbf{p}(\mathbf{q}_b)\| \quad (1.9)$$

This distance is called **displacement metric**. Notice how in the definition we are writing $\mathbf{p} \in \text{robot}$, by letting robot be the set of all the points in the structure of the manipulator, which clearly depend on the current configuration. This kind of metric works under the assumption that, if the region occupied by the robot is similar in two different postures, then the distance between their respective configurations is small:

$$B(\mathbf{q}_a) \simeq B(\mathbf{q}_b) \implies d(\mathbf{q}_a, \mathbf{q}_b) \simeq 0. \quad (1.10)$$



To compute the distance in Equation (1.9), we should check each point of the robot, and in general this is an uncountably infinite set. If the links are polygonal shapes, the problem becomes finite, since it is possible to find the maximum distance between any pair of polytopes in the three dimensional space.

A more computationally feasible approach is to check only for a finite set of points in the robot structure, called **control points**. Let

$$P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_c\} \quad (1.11)$$

be a set of c points of the robot (in general, they all depend on the configuration $\mathbf{q}$). We compute the distance between two postures in the following way:

$$d(\mathbf{q}_a, \mathbf{q}_b) = \max_{1 \leq i \leq c} \|\mathbf{p}_i(\mathbf{q}_a) - \mathbf{p}_i(\mathbf{q}_b)\|. \quad (1.12)$$

Not all the points of a robot are feasible control points, for example, the base of a manipulator usually doesn't change with the configuration. P is a set of *proper control points* if the function d as defined in Equation (1.12) satisfies the properties of a distance given in Definition 1.1.4.

Observation 1.1.1 : Approximation for Local Motion

Since a manifold is locally homeomorphic to $\mathbb{R}^n$, for sufficiently close configurations we can approximate the distance on the manifold (assuming it is smooth) with the Euclidean distance.

Observation 1.1.2 : Prismatic Joints

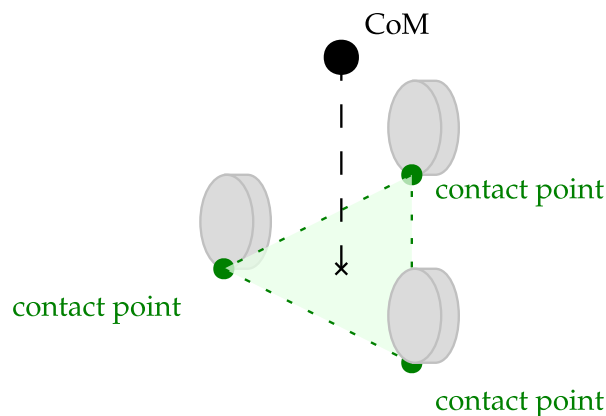
The configuration space of a robot with *only* prismatic joints is Euclidean.

1.2 Mechanics of Mobile Robots

The main sources of movement for a mobile robot are:

- **Wheels:** wheeled robots usually consists of one single rigid body, with the wheels in contact with ground.
- **Feet:** legged robots, several rigid bodies articulated through joints.

The wheels are in contact with the ground, and this introduce some constraints. Each wheel defines a point of contact with the ground, we call the **support polygon** the convex hull of all the contact point. A wheeled robot is in **static balance** if the projection on the ground of its center of mass, falls inside the support polygon. Of course a robot must be balanced in order to not fall to the ground.



The convex hull of two points is a line, hence a robot with only two wheels can't be easily balanced in a static fashion.

There are three main types of wheels:

- **Fixed Wheels:** They have a fixed orientation: Can roll but can't steer on their z-axis (see Figure 1.5). They are usually actuated, if passive, are used for balance and offer a contact point.
- **Steerable Wheels:** They can steer and are used for orientation, hence they are typically active (i.e. the front wheels of a car).
- **Caster Wheels:** They have a variable orientation but they are typically not actuated. Their axis of orientation are not aligned, this property makes the wheels align automatically with the direction of motion, hence they are "invisible" kinematically, but offer a contact point.

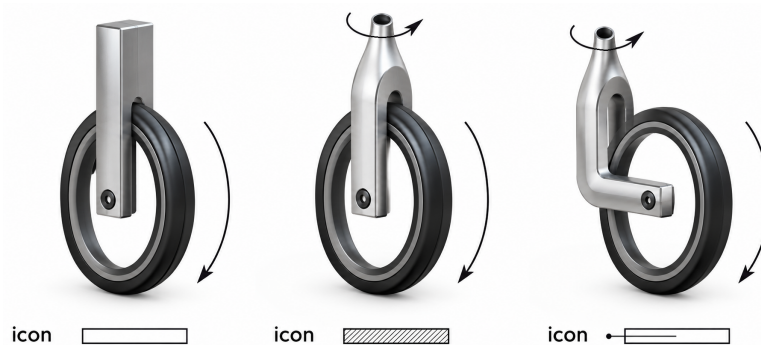
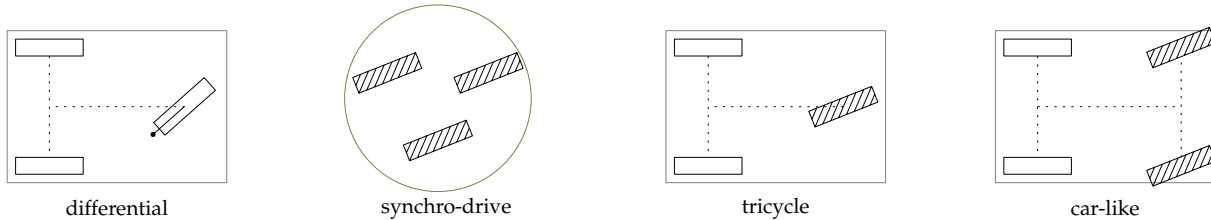


Figure 1.5: Fixed, steerable and caster wheels.

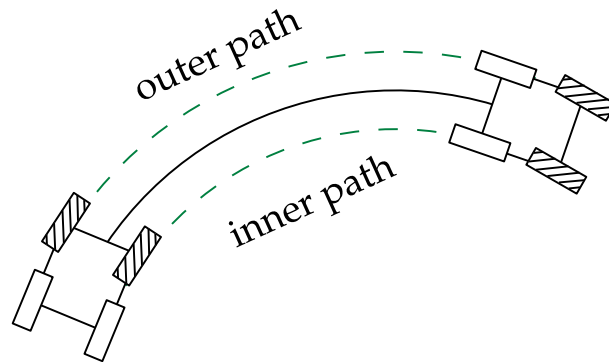
1.2.1 Main Kinematic Structures

The simplest wheeled robot that we can have is the **differential drive** robot: it uses two aligned fixed wheels that are independently actuated, and one caster wheel added for balance. The **synchro-drive** mobile robots have three steerable wheels and two actuators: one controls simultaneously the orientation of all the wheels, and the other rolls them together. The base over the wheels hence will never change orientation. This type of configuration is preferable when the body to move is particularly heavy.



The tricycle has two fixed wheels in the back and one actuated steerable wheel in the front, the car-like is similar, but with two steerable wheels.

Consider the following path taken by a car-like robot:



In the same time interval, the wheels on the outer path travels a longer distance with respect to the wheels on the inner path, hence their rotation speed must be higher. In order to avoid using two distinct actuators for the front wheels we use a device called **differential**.

Definition 1.2.1 : Differential

A mechanical differential is a gear assembly (usually using bevel or planetary gears) connected to a single drive motor. It splits power to two wheels on the same axle while allowing them to rotate at different speeds. When a robot with a single motor turns, the outside wheel must travel a larger arc than the inside wheel. The differential lets the wheels spin at their own natural speeds, preventing tire slip, scrubbing, and mechanical strain on the drivetrain.

1.3 Constraints and Mobility

The presence of the wheels in contact with the ground introduce a series of *geometric* and *kinematic* constraints. We will also see the relationship between these two types of constraints and the implications in mobility.

1.3.1 Geometric Constraints

We have seen how the configuration $\mathbf{q}$ is in general a point of a manifold $\mathcal{C}$, that is locally homeomorphic to $\mathbb{R}^n$, with n being the dimensionality of the configuration space $\mathcal{C}$. We will focus on equality time-invariant constraints. All the geometric constraints are expressed in the following form:

$$h(\mathbf{q}) = 0 \quad (1.13)$$

and they depend only on the configuration. Usually we have $k < n$ geometric constraints, and they are reflected on the degrees of freedom of a robot:

$$h_1(\mathbf{q}) = 0 \quad (1.14)$$

$$\vdots \quad (1.15)$$

$$h_k(\mathbf{q}) = 0. \quad (1.16)$$

If the configuration space is a 2-manifold, one single geometric constraint will reduce the degrees of freedom of the robot, constraining it to move on a 1-dimensional manifold (a curve).

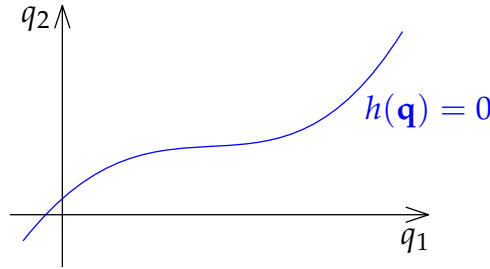


Figure 1.6: If the configuration space is $\mathbb{R}^2$, one constraint will let the configurations lie on a curve in the plane.

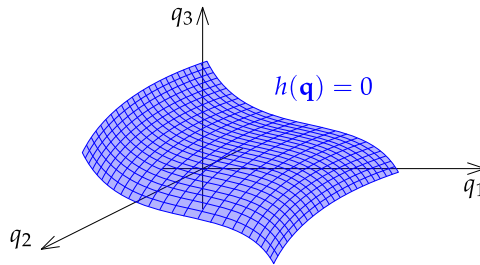


Figure 1.7: If the configuration space is $\mathbb{R}^3$, one constraint will let the configurations lie on a surface in the space.

In general, a robot with a configuration space that is an n -manifold with k independent geometric constraints, will move on an $(n - k)$ -manifold. Instead of using the vector $\mathbf{q}$, we can reduce the number of coordinates in many ways, like for one constraint:

$$h(\mathbf{q}) = 0 \quad (1.17)$$

we can solve the equation for a coordinate q_i :

$$q_i = \tilde{h}(q_1, \dots, q_{i-1}, q_{i+1}, \dots, q_n). \quad (1.18)$$

For example, consider a 2 degrees of freedom robot with the following constraint:

$$q_1^2 + q_2^2 = 1 \quad (1.19)$$

the configuration space is limited to the unit circle in $\mathbb{R}^2$ (1-manifold), we solve the constraint for q_2 :

$$q_2 = \pm \sqrt{1 - q_1^2}. \quad (1.20)$$

Actually this is not a good way to choose a coordinate, since the $\pm$ term prevents us from assigning a single value of q_2 to each value of q_1 , requiring a local resolution. For this example of the circle it is better to use the arc length $s \in [0, 2\pi)$ as the single coordinate.

Observation 1.3.1 : Sub-Manifold

If the configuration space $\mathcal{C}$ is an n -manifold, the k constraints define a $(n - k)$ -manifold embedded in $\mathcal{C}$, that we also call sub-manifold.

1.3.2 Kinematic Constraints

A kinematic constraint is local, it prevents the configuration to move towards some directions. A general constraint of this kind is in the form:

$$a(\mathbf{q}, \dot{\mathbf{q}}) = 0 \quad (1.21)$$

it depends on the state of the robot. In every given configuration $\mathbf{q}^*$, we can immediately infer from $a(\mathbf{q}^*, \dot{\mathbf{q}}) = 0$ which velocities are allowed. All the relevant kinematic constraints in the robotic domain are in **Pfaffian form**:

$$\mathbf{a}^T(\mathbf{q})\dot{\mathbf{q}} = 0 \quad (1.22)$$

with $\mathbf{a}$ being an n dimensional vector that depends on the configuration. The Pfaffian form expresses a condition of orthogonality: in a given configuration the velocity must be orthogonal to the vector $\mathbf{a}$. More kinematic constraints can be arranged in matrix form:

$$\begin{pmatrix} \mathbf{a}_1^T(\mathbf{q}) \\ \mathbf{a}_2^T(\mathbf{q}) \\ \vdots \\ \mathbf{a}_k^T(\mathbf{q}) \end{pmatrix} \dot{\mathbf{q}} = \mathbf{0} \implies A(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0} \implies \dot{\mathbf{q}} \in \mathcal{N}(A). \quad (1.23)$$

The admissible velocities are the ones that lie in the null space of A^1 , hence they lie in a linear space.

Observation 1.3.2 : Relationship Between Geometric and Kinematic Constraints

Any geometric constraint implies a kinematic constraint.

Consider a geometric constraint

$$h(\mathbf{q}) = 0 \quad (1.24)$$

this constraint must be satisfied as the time goes on, hence its derivative should also satisfy the following relation:

$$\dot{h}(\mathbf{q}) = 0 \quad (1.25)$$

by applying the chain rule:

$$\dot{h}(\mathbf{q}) = \frac{\partial h}{\partial \mathbf{q}} \dot{\mathbf{q}} = 0 \quad (1.26)$$

$\frac{\partial h}{\partial \mathbf{q}}$ is a row vector, the transpose of the gradient of h :

$$(\nabla_{\mathbf{q}} h)^T \dot{\mathbf{q}} = 0 \quad (1.27)$$

this is exactly a kinematic constraint in Pfaffian form. The gradient of h evaluated on a point is the vector orthogonal to the manifold on which the configuration is constrained to move, hence the resulting kinematic constraint is preventing the robot from going out of the sub-manifold defined by the geometric constraints.

¹Sometimes, we let the dependencies of a term implicit.

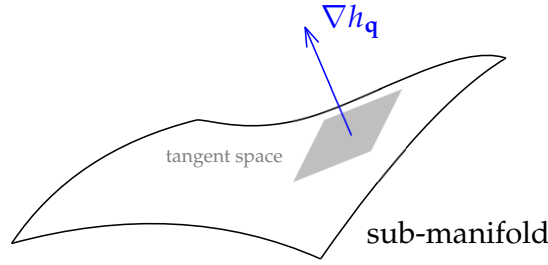


Figure 1.8: The admissible velocities are the one that lie on the tangent space of the sub-manifold in the given configuration.

We have seen how a geometric constraint always implies a kinematic one. The opposite is not generally true, consider a generic kinematic constraint:

$$\mathbf{a}^T(\mathbf{q})\dot{\mathbf{q}} = 0 \quad (1.28)$$

this implies a geometric constraint if there exists a function $h(\mathbf{q})$ such that

$$\nabla_{\mathbf{q}}h = \mathbf{a} \quad (1.29)$$

hence $\mathbf{a}(\mathbf{q})$ should be *integrable*.

Definition 1.3.1 : Holonomic Constraint

An integrable kinematic constraint is called **holonomic**, and it defines a geometric constraint.

What conditions determine if a constraint is integrable or not? In general, the conditions are stated in the *Frobenius Theorem*, we will not discuss this result, which belongs to the domain of differential geometry. We will determine if a given constraint is holonomic or not by a different way.

Consider the following kinematic constraint for a 2-dimensional robot:

$$q_1\dot{q}_1 + q_2\dot{q}_2 = 0 \quad (1.30)$$

is this integrable? The answer is yes. The resulting geometric constraint is

$$q_1^2 + q_2^2 = c, \quad c > 0. \quad (1.31)$$

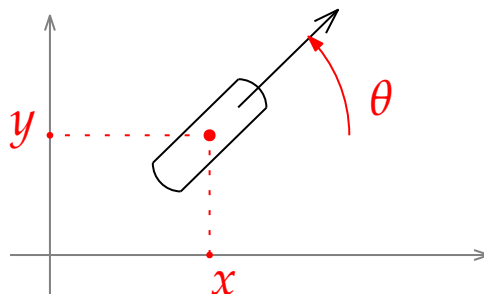
In general, in Equation (1.29) we can add a scalar integrating factor that depend on the configuration:

$$\frac{\partial h}{\partial \mathbf{q}} = \gamma(\mathbf{q})\mathbf{a}^T(\mathbf{q}) \quad (1.32)$$

since the right-hand side is zero, γ can be dropped.

1.3.3 Non-Holonomic Constraints

A non-holonomic constraint defines a local limitation, it stops the robot from moving towards a certain direction when it is in that specific configuration, it does not prevent the robot from reaching a configuration by following an allowed path. The simplest example of a system with a non-holonomic constraint is a rolling disk.



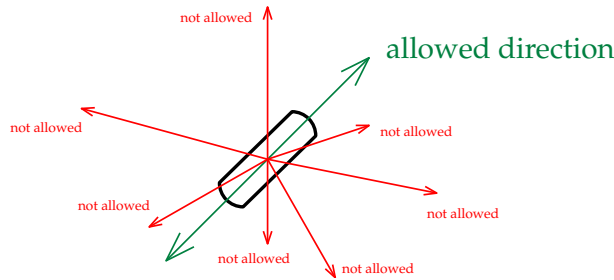
The configuration is given by the vector $(x \ y \ \theta)^T$, θ is its orientation with respect to the horizontal axis. It is clear that the disk in pure rolling (no slippage) can move *only* in the direction that is pointing (by rolling), hence can't move laterally. This constraint is expressed as follows:

$$\sin \theta \dot{x} - \cos \theta \dot{y} = 0 \quad (1.33)$$

or in vector form:

$$(\sin \theta \quad -\cos \theta \quad 0) \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = 0 \quad (1.34)$$

$\mathbf{n}^T = (\sin \theta \quad -\cos \theta)$ is the vector *normal* to the direction. Any velocity that has a component on the line spanned by $\mathbf{n}$ is not allowed.



This constraint does not imply any global limitation, any configuration is still reachable, hence it is a non-holonomic constraint. To prove this, instead of using the Frobenius theorem, we give a non-formal proof by showing that, for any pair of configurations $\mathbf{q}_s, \mathbf{q}_g$, there exists a trajectory in the configuration space that connects these two configurations without violating the kinematic constraint. Let

$$\mathbf{q}_s = \begin{pmatrix} x_s \\ y_s \\ \theta_s \end{pmatrix}, \quad \mathbf{q}_g = \begin{pmatrix} x_g \\ y_g \\ \theta_g \end{pmatrix}. \quad (1.35)$$

1. $\dot{\theta}$ is not constrained, first of all we move the orientation from θ_s to θ^* , where θ^* is defined as the angle such that

$$(\sin \theta^* \quad -\cos \theta^*) \begin{pmatrix} x_g - x_s \\ y_g - y_s \end{pmatrix} = 0. \quad (1.36)$$

2. Then, we can move on the direction $(x_g - x_s \ y_g - y_s)^T$, in this way we can reach the desired position $(x_g \ y_g)^T$.
3. At the end, we move the orientation from θ^* to θ_g .

This is called **universal maneuver**.

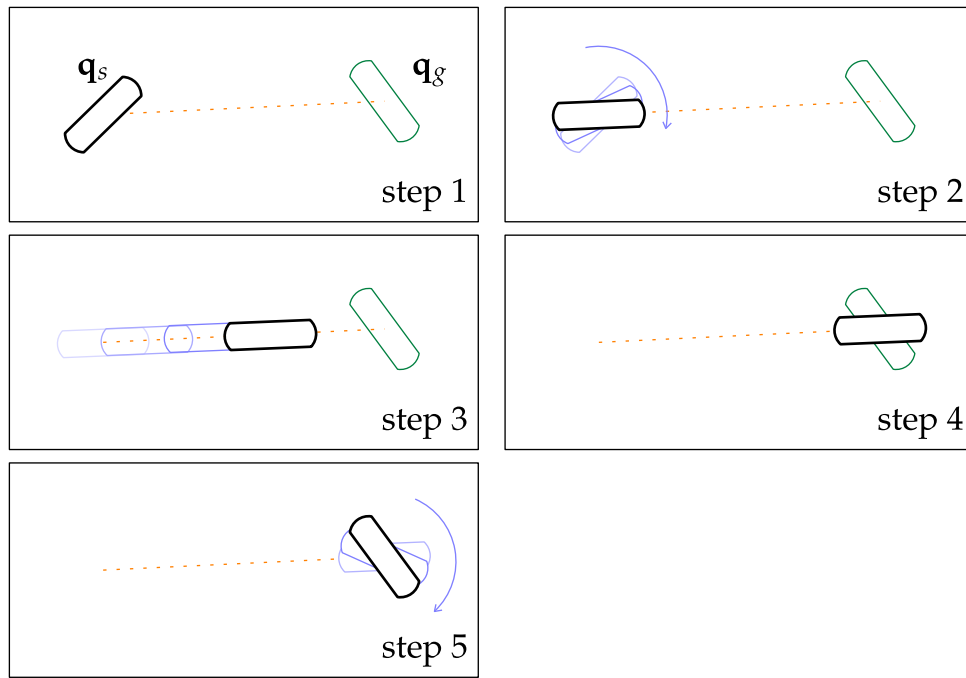
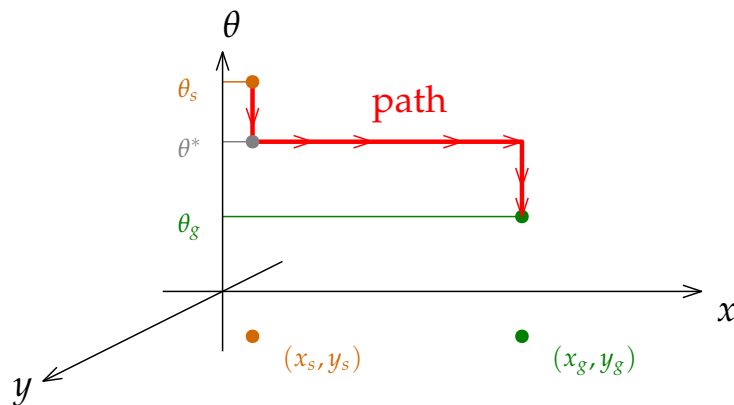


Figure 1.9: The universal maneuver in five steps.

There are no global mobility limitations. We can also observe this trajectory in the configuration space:



Note how for simplicity we have drawn the configuration space as Euclidean. Of course it is not Euclidean because $\theta \in SO(2)$.